Diagramas UML: Clases y Secuencia

Ing. Andrés Eduardo Paternina Ariza Esp.
Universidad del Magdalena
apaternina@unimagdalena.edu.co

¿Por qué UML?

UML (Unified Modeling Language) es un **lenguaje gráfico estándar** para representar el diseño de un sistema. Un diagrama **lo entienden todos en el equipo**, sin importar el lenguaje de programación.

ANTES DE PROGRAMAR



Comunicar el diseño

Acordar en equipo cómo se estructura el sistema antes de escribir una sola línea de código. Evita rehacer trabajo.

DESPUÉS DE PROGRAMAR

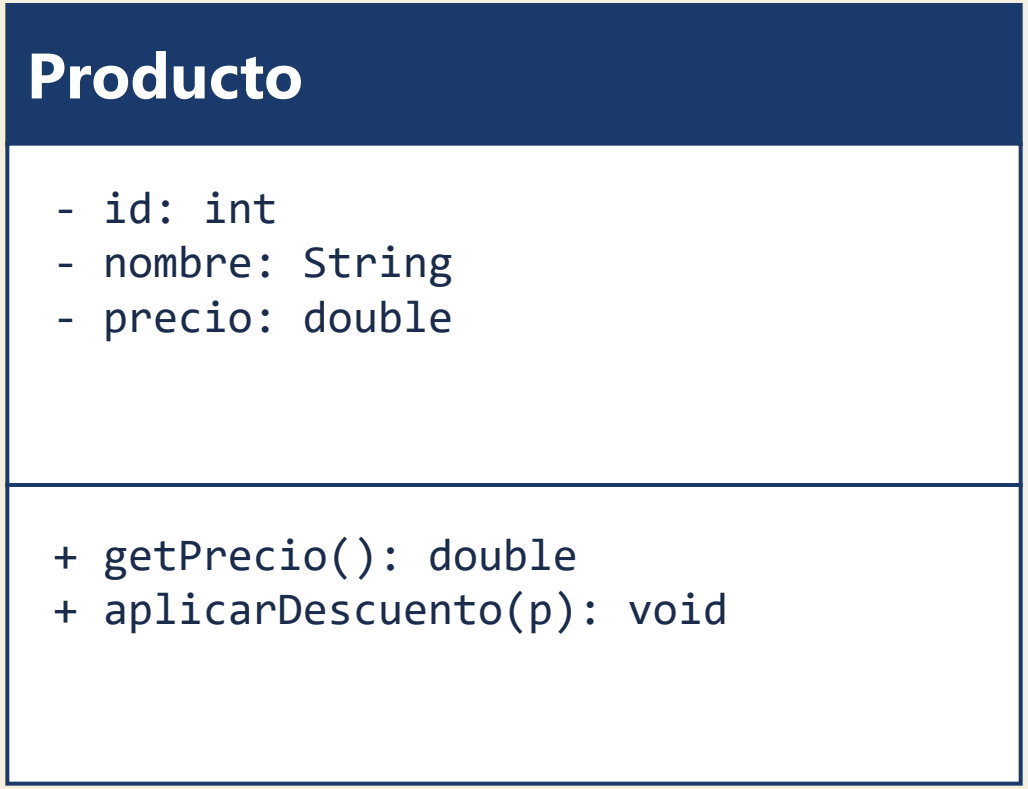


Documentar lo existente

Explicar cómo funciona un sistema ya construido de un vistazo, sin obligar a nadie a leer todo el código.

La notación de una clase

Un rectángulo dividido en tres compartimentos



TRES COMPARTIMENTOS

1. **Nombre de la clase**
2. **Atributos — los datos que guarda**
3. **Métodos — lo que sabe hacer**

VISIBILIDAD

```
+   público – accesible desde fuera
-   privado – solo dentro de la
clase
#   protegido – clase y subclases
```

Relaciones entre clases

HERENCIA

Herencia

"es un". Flecha con triángulo vacío hacia la clase padre.



ASOCIACIÓN

Asociación

"usa / conoce a". Una línea simple entre dos clases.

MULTIPLICIDAD

Multiplicidad

Números en los extremos: cuántos objetos participan.



INTERFAZ

Interfaz

"implementa". Línea punteada con triángulo vacío hacia la interfaz.



Enunciado

Una **empresa** tiene una **sede principal** y cuenta con varios **empleados**. Cada empleado tiene un **nombre, un documento y un cargo.**

La empresa vincula a su personal de dos formas: **empleados fijos** y **contratistas**, que además de los datos comunes cuentan con atributos propios.

Empresa y sede

- Una empresa tiene una sola sede
- Cada sede pertenece a una sola empresa
- Una empresa tiene muchos empleados
- Cada empleado trabaja para una sola empresa

Empleado fijo

- Nombre, documento y cargo
- Salario mensual
- Fecha de ingreso
- Pago = salario mensual + prestaciones

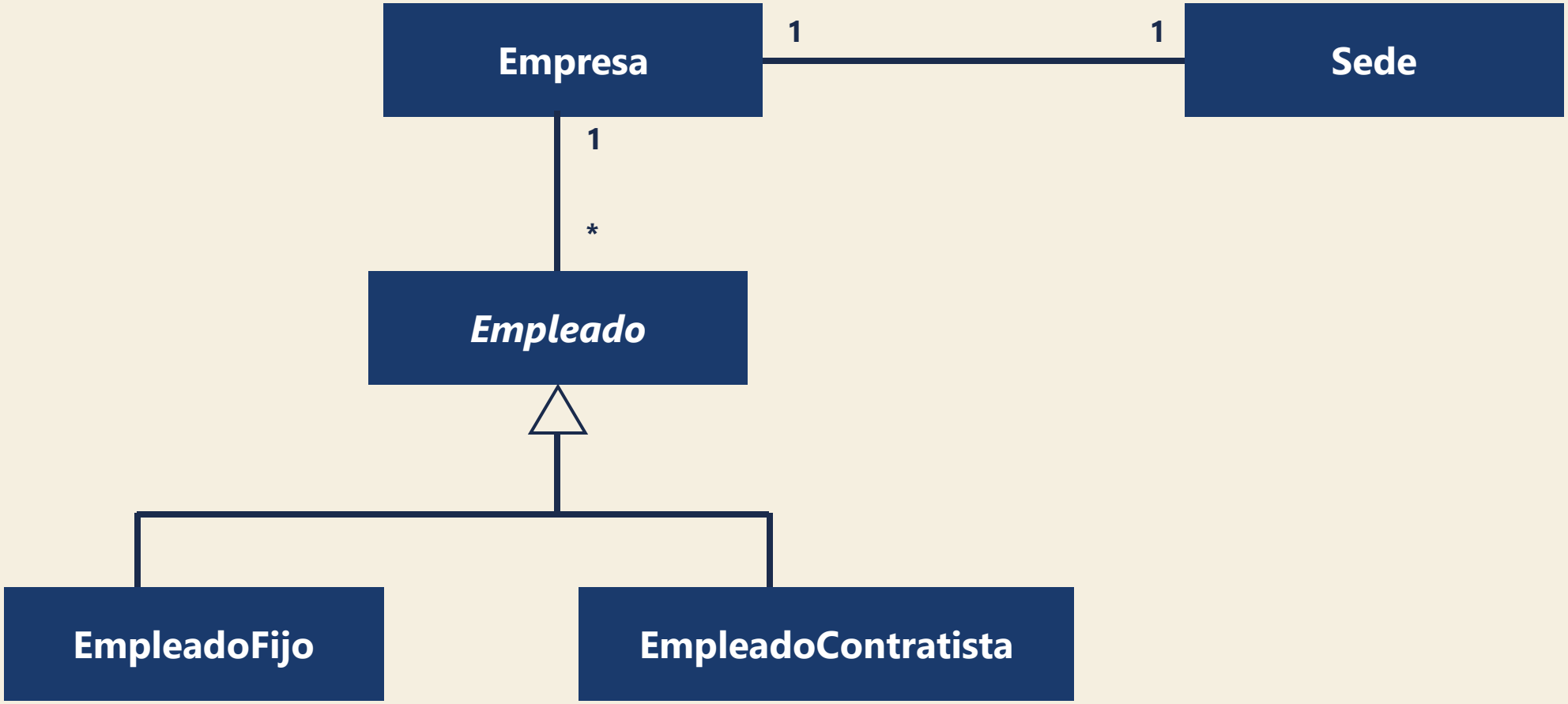
Empleado contratista

- Nombre, documento y cargo
- Valor por hora
- Horas trabajadas en el mes
- Pago = valor por hora × horas trabajadas

No se pueden registrar **empleados genéricos**, solo fijos o contratistas. Todos deben poder **calcular su pago**, pero cada tipo lo hace **a su manera**.

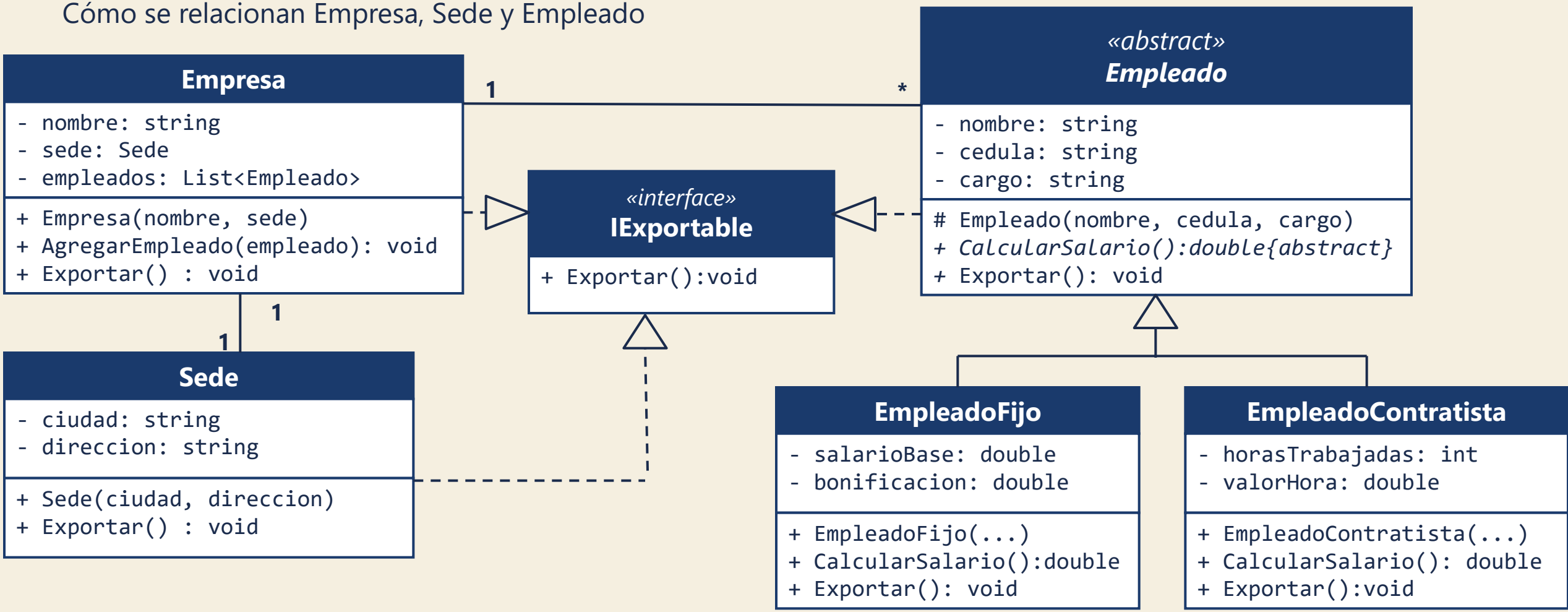
Ejemplo

Cómo se relacionan Empresa, Sede y Empleado



Ejemplo

Cómo se relacionan Empresa, Sede y Empleado



¿Qué es un diagrama de secuencia?

Un **diagrama de secuencia** es un tipo de diagrama UML de comportamiento que muestra **cómo interactúan los objetos entre sí a lo largo del tiempo**, siguiendo el orden cronológico de las acciones que se realizan.

QUÉ MUESTRA



Los mensajes entre objetos

Quién llama a quién, qué responde cada objeto y en qué orden ocurre cada paso.

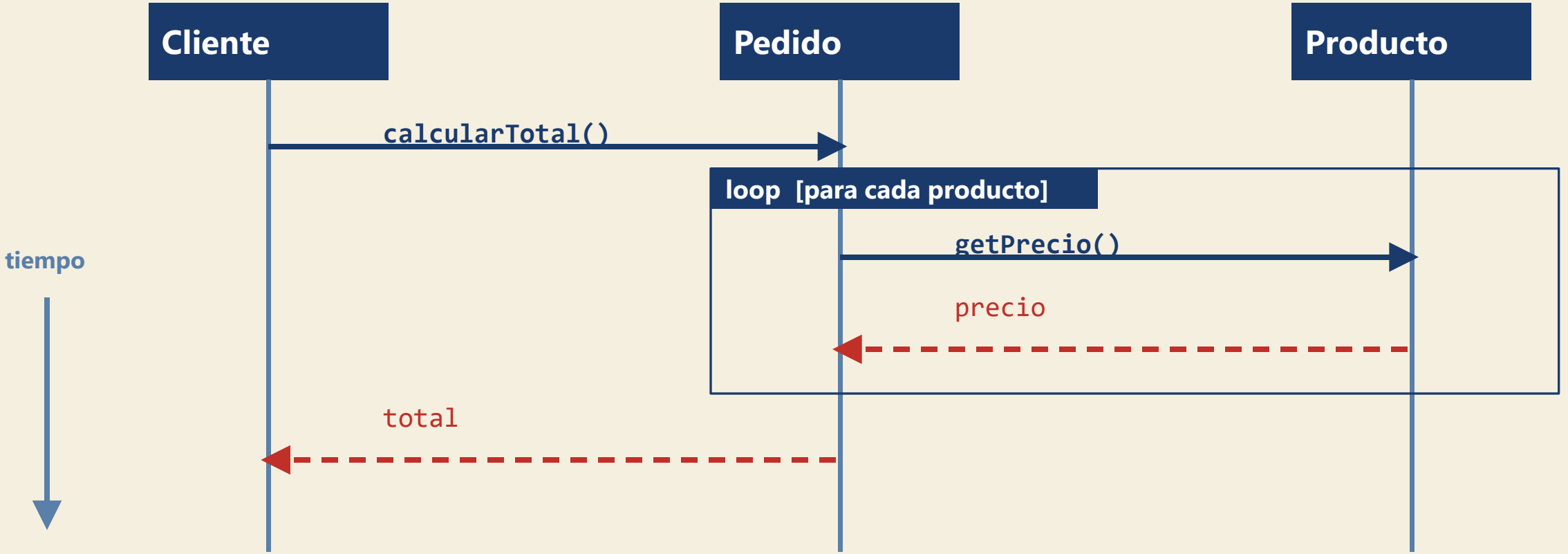
PARA QUÉ SIRVE



Seguir un proceso paso a paso

Recorrer un caso de uso completo, como hacer un pedido, y detectar llamadas que sobran o faltan.

Notación de secuencia



Enunciado

Una **empresa** tiene una **sede principal** y cuenta con varios **empleados**. Cada empleado tiene un **nombre, un documento y un cargo.**

La empresa vincula a su personal de dos formas: **empleados fijos** y **contratistas**, que además de los datos comunes cuentan con atributos propios.

Empresa y sede

- Una empresa tiene una sola sede
- Cada sede pertenece a una sola empresa
- Una empresa tiene muchos empleados
- Cada empleado trabaja para una sola empresa

Empleado fijo

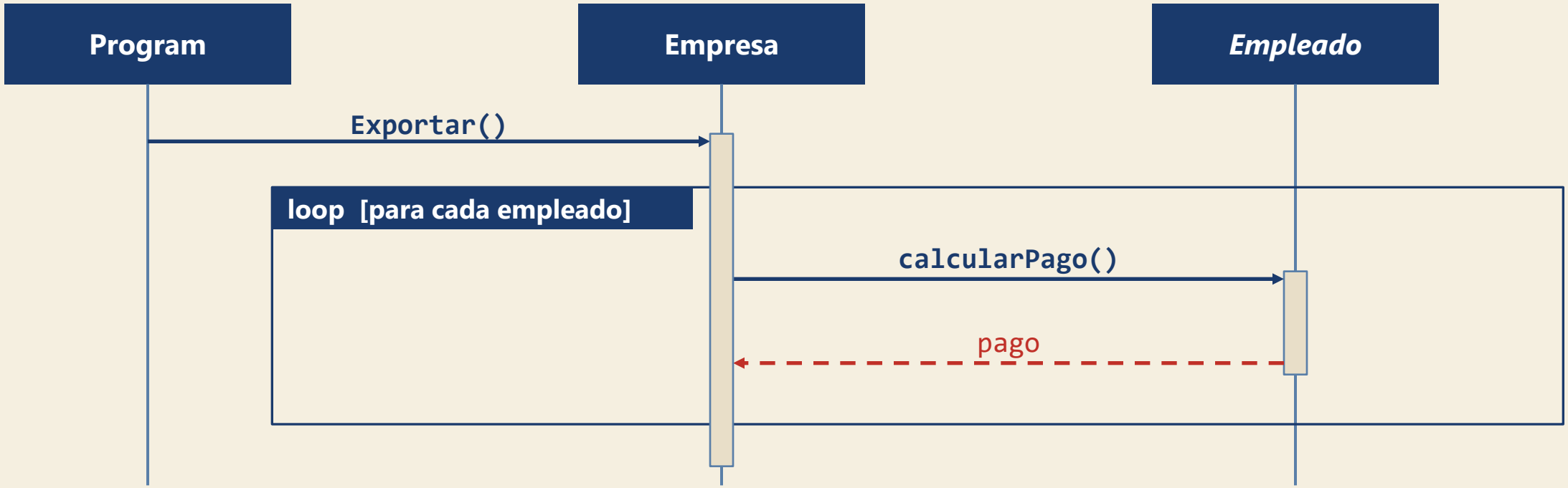
- Nombre, documento y cargo
- Salario mensual
- Fecha de ingreso
- Pago = salario mensual + prestaciones

Empleado contratista

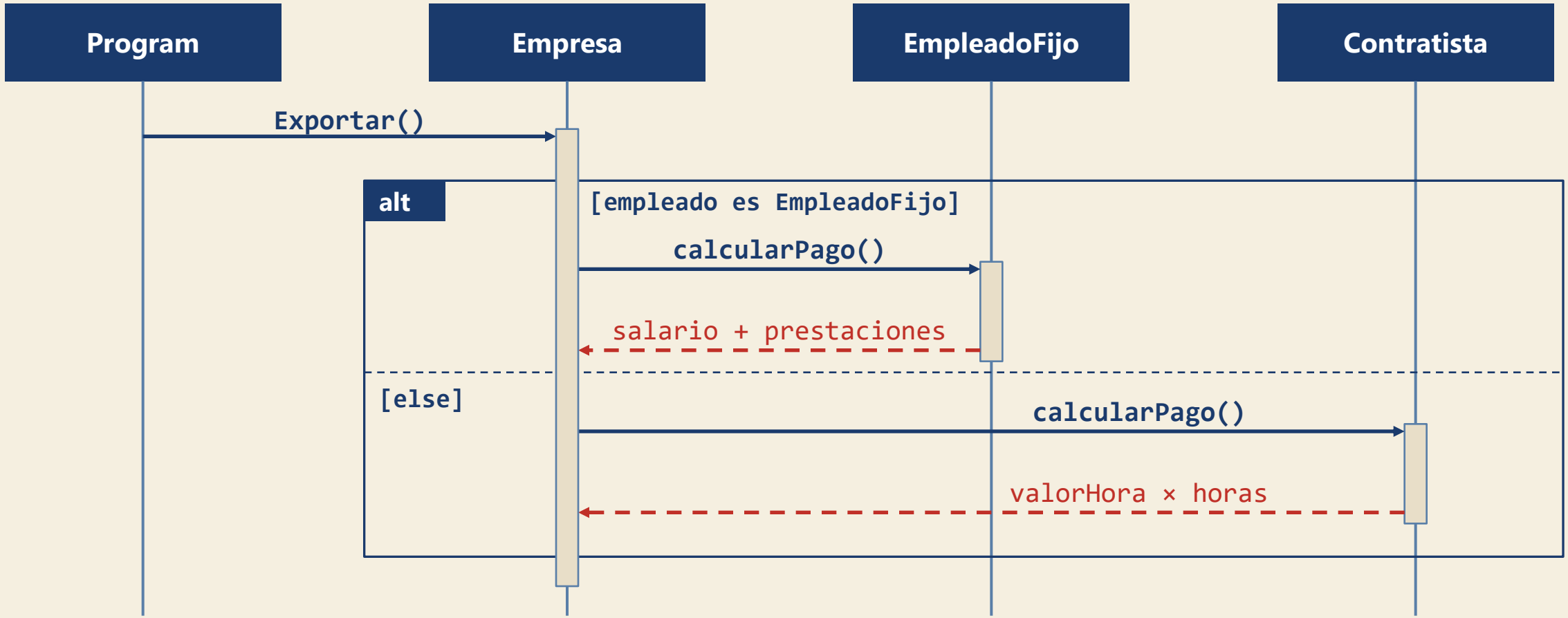
- Nombre, documento y cargo
- Valor por hora
- Horas trabajadas en el mes
- Pago = valor por hora × horas trabajadas

No se pueden registrar **empleados genéricos**, solo fijos o contratistas. Todos deben poder **calcular su pago**, pero cada tipo lo hace **a su manera**.

Secuencia: cálculo del pago con Empleado general



Secuencia: cómo se calcula el pago



Secuencia: cómo se agrega un empleado



Taller práctico: diagrama de clases

A partir del **código en C#** del centro veterinario, elaboren el **diagrama de clases** del sistema.

Trabajen sobre el código tal como está: no agreguen clases ni métodos que no existan.

1. Clases

- Identificar cada clase del código
- Marcar clases abstractas e interfaces

2. Atributos y métodos

- Con su tipo y visibilidad
- Métodos abstractos en cursiva

3. Relaciones

- Herencia e implementación de interfaces
- Asociaciones con su multiplicidad

Taller práctico: diagramas de secuencia

Con el mismo código, elaboren un **diagrama de secuencia** para cada una de estas tres funcionalidades.

1. Registrar una atención

- Un registro recibe una nueva atención
- Mostrar qué ocurre si el registro está abierto y si está cerrado

2. Calcular el total a pagar

- El registro obtiene el costo de cada atención y acumula los valores
- Aplica la modalidad de cobertura

3. Cerrar un registro

- El registro comprueba si existen atenciones antes de permitir su cierre